

AI IN ACTION - Day 1

AI & LLM Foundation

Bạn đang dùng AI mỗi ngày — nhưng thực sự bên trong nó đang làm gì?

Agenda

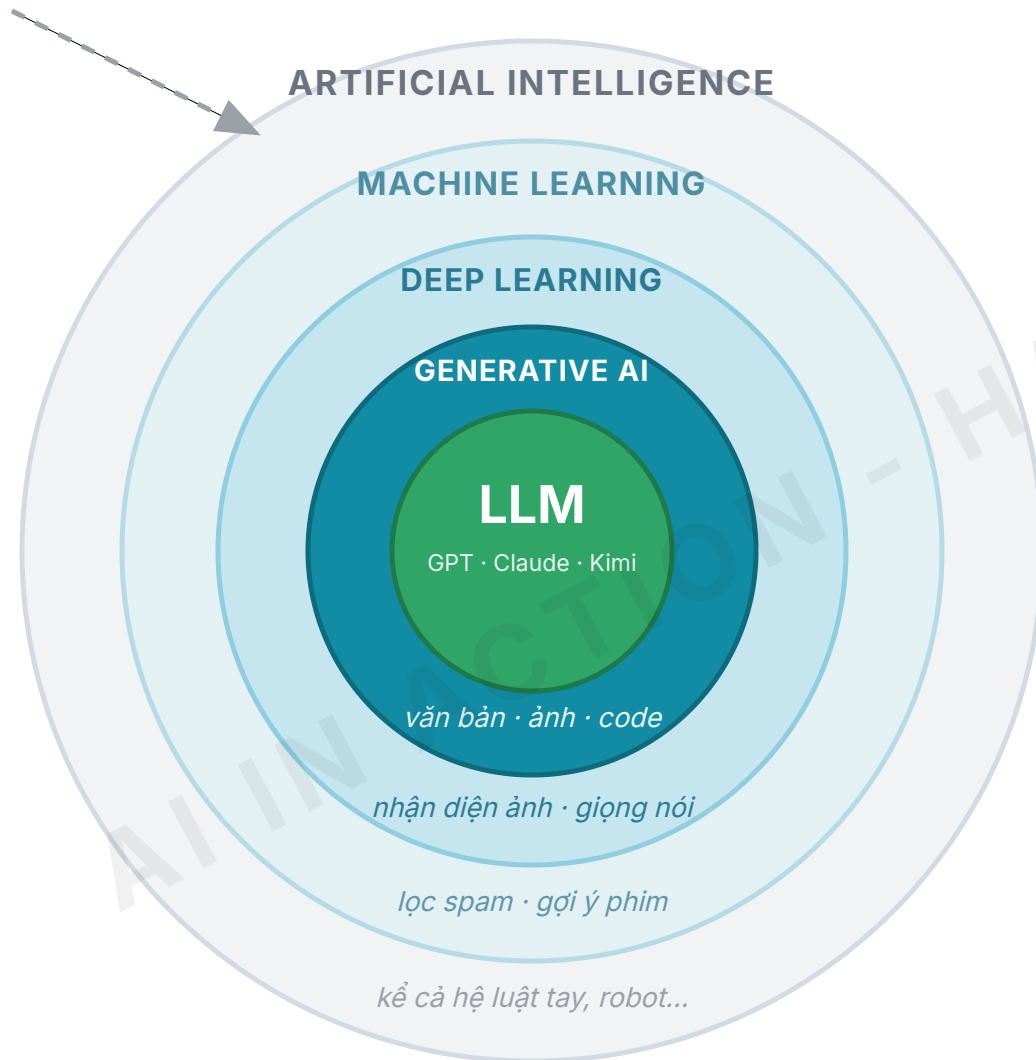
- Bức tranh AI & các tầng của AI
- Lịch sử AI 70 năm
- Bên trong LLM: cơ chế vận hành
- Từ LLM đến AI Agent
- Landscape: model hôm nay & cuộc đua hiện tại
- Chọn model & chi phí token
- Gọi API lần đầu
- Tổng kết — những ý để mang về

AI & LLM Foundation

Từ "nghe AI" đến "gọi AI" trong một ngày

AI, ML, Deep Learning, GenAI, LLM — nằm ở đâu trong cùng một hệ?

từ rộng đến hẹp



- **AI** — chiếc ô lớn nhất: mọi hệ thống có yếu tố “thông minh”.
- **Machine learning** — học từ dữ liệu thay vì viết luật tay.
- **Deep learning** — mạng nơ-ron nhiều tầng tự học đặc trưng.
- **Generative AI** — sinh nội dung mới: văn bản, ảnh, code.
- **LLM** — model nền chuyên ngôn ngữ, tim của làn sóng hiện nay.

LLM không phải toàn bộ AI — nhưng nó là tầng nền của gần hết trải nghiệm AI bạn dùng hôm nay

Ba nhóm AI chính: phân loại · sinh nội dung · hành động

Discriminative AI

Giỏi **phân loại, dự đoán**: lọc spam, phát hiện gian lận, nhận diện ảnh.

Input → một nhãn, một con số

Generative AI

Sinh ra thứ mới: văn bản, ảnh, code.
ChatGPT, Claude, Midjourney.

Prompt → nội dung mới

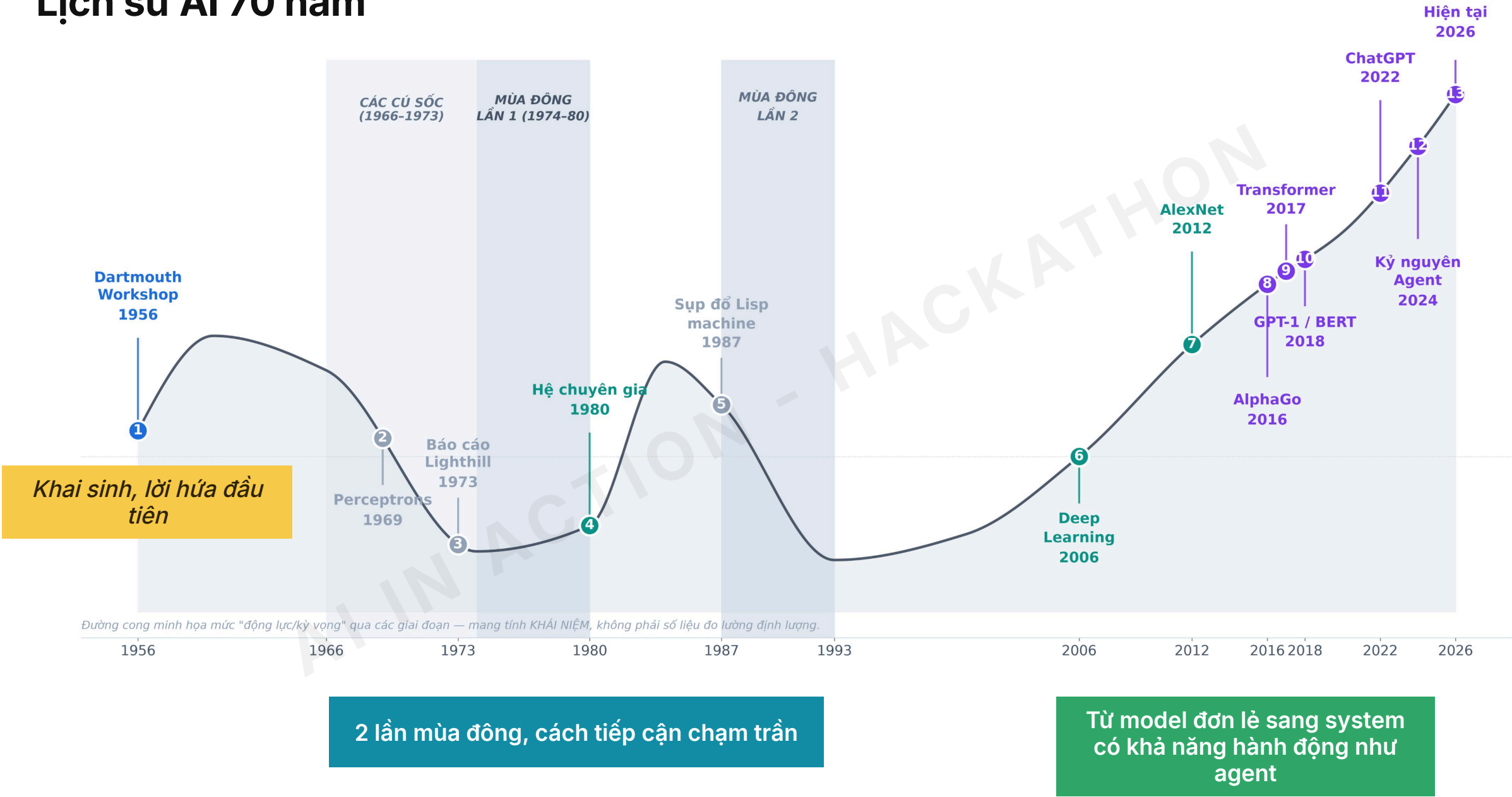
Agentic AI

Nhận **mục tiêu** rồi tự làm nhiều bước:
lập kế hoạch, dùng công cụ, hành động.

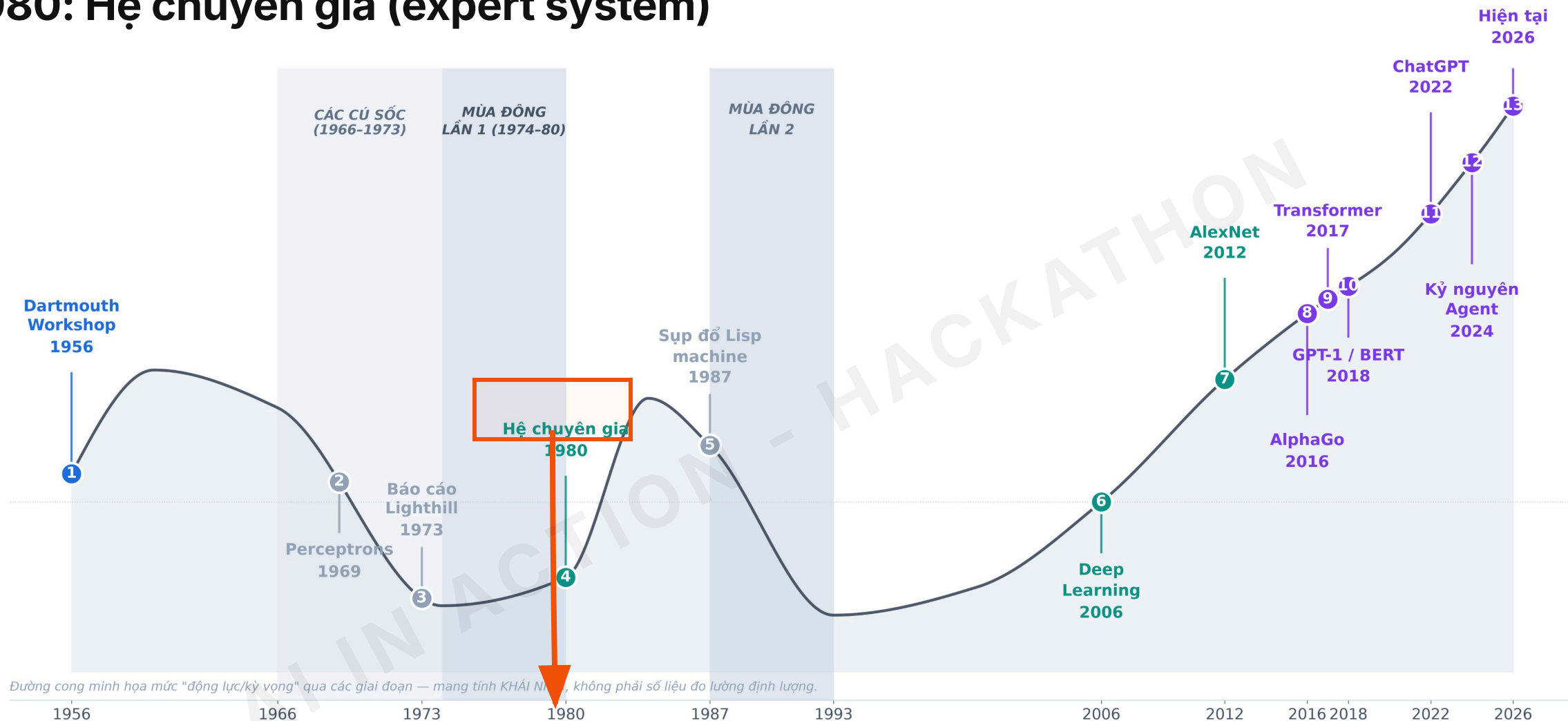
Goal → *Plan* → *Action*

LLM là engine chung của cả Generative lẫn Agentic — cuối buổi sáng mình sẽ thấy agent khác LLM ở đâu

Lịch sử AI 70 năm



1980: Hệ chuyên gia (expert system)

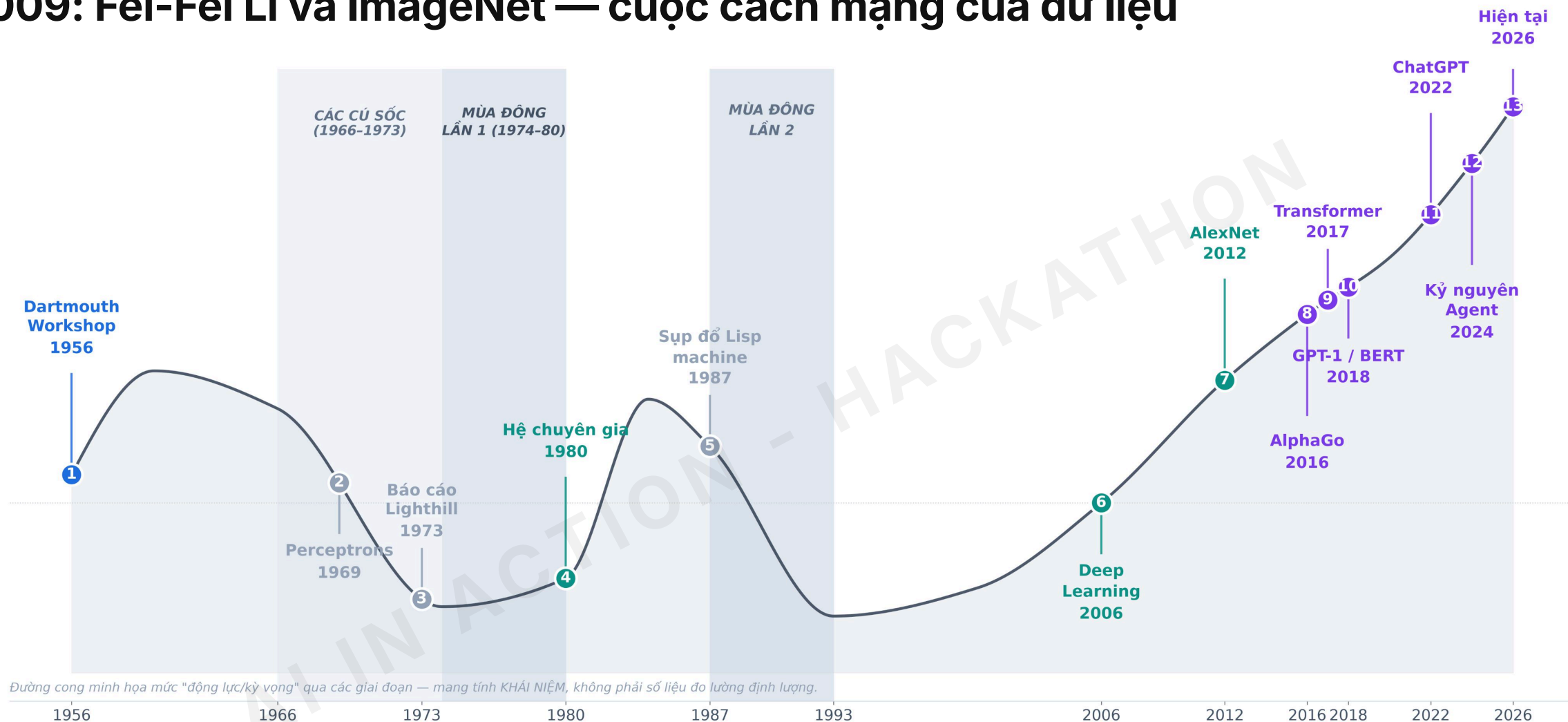


Đặt lại vấn đề: *"Nếu AI chỉ giải thật tốt một loại bài toán chuyên môn hẹp thì sao?"*

→ Sự ra đời của **expert systems**

AI đổi chiến lược: thôi theo đuổi trí tuệ tổng quát và **tập trung giải thật tốt một miền hẹp** bằng cách mã hóa tri thức chuyên gia thành luật

2009: Fei-Fei Li và ImageNet — cuộc cách mạng của dữ liệu

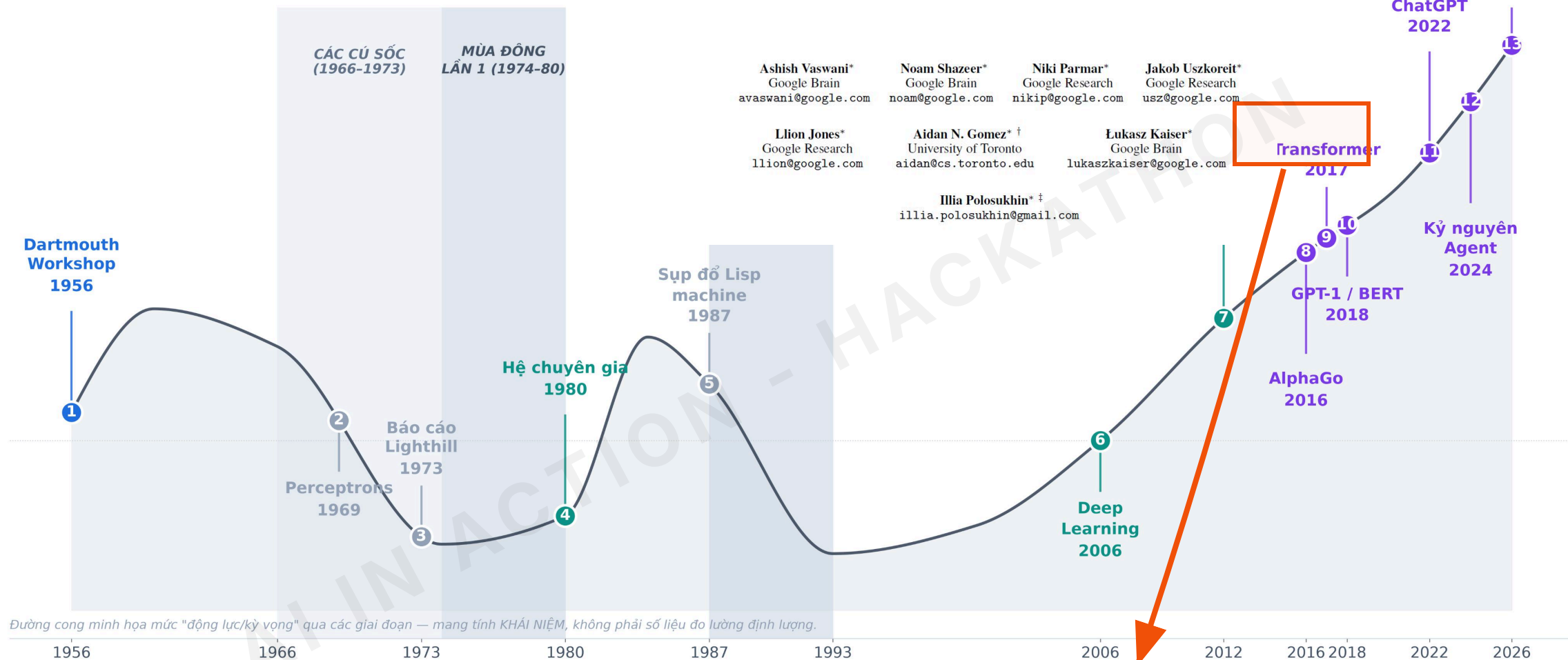


Trong khi cả ngành chạy theo thuật toán thông minh hơn, Fei-Fei Li chọn con đường khác: **xây bộ dữ liệu lớn hơn** — 14 triệu ảnh được gán nhãn tay, hơn 20.000 loại vật.

Ba năm sau, chính bộ dữ liệu đó là sân khấu cho cú nổ AlexNet 2012 → bài học định hình cả kỷ nguyên: **đôi khi dữ liệu tốt hơn đánh bại thuật toán khôn hơn.**

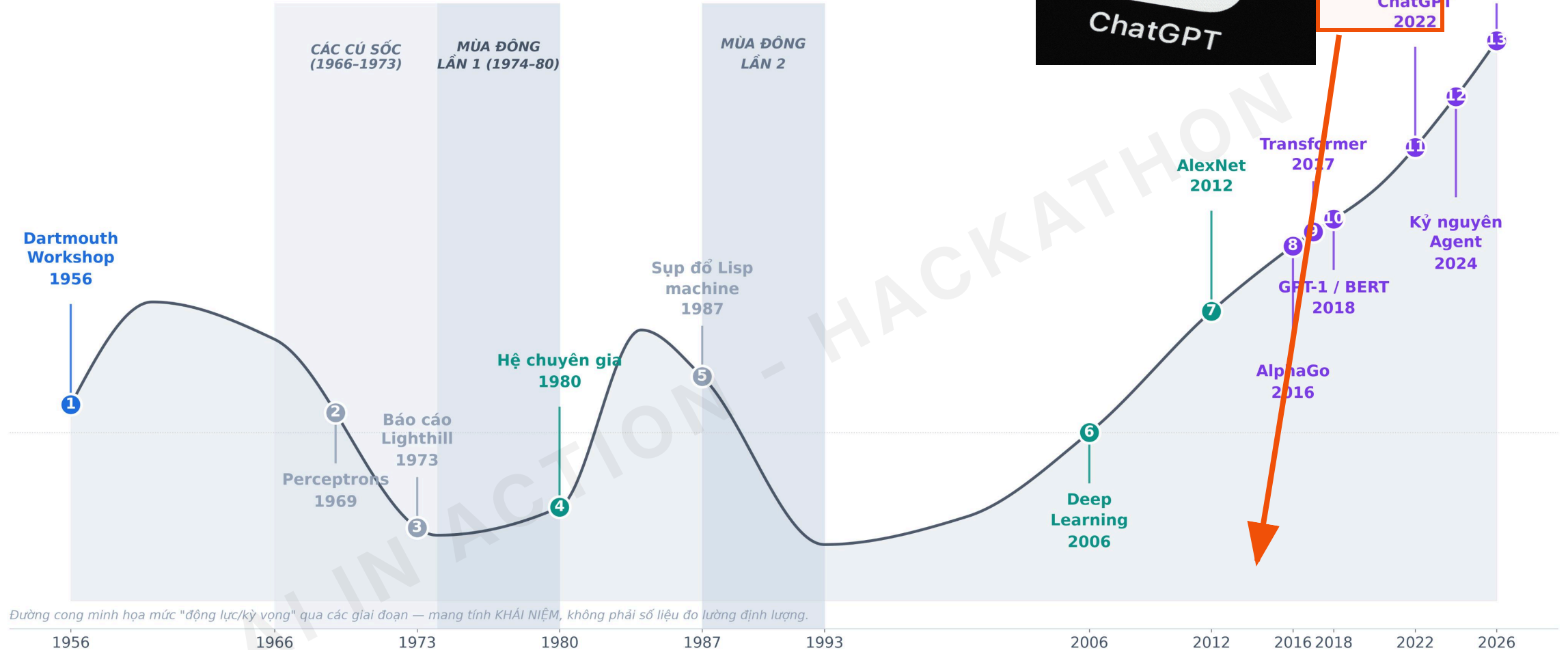
2017: Transformer

Attention Is All You Need



Transformer là bước ngoặt vì nó cho mô hình hiểu ngôn ngữ theo cách linh hoạt hơn: **mỗi từ có thể nhìn sang những từ quan trọng khác** trong cả câu, thay vì chỉ đi tuần tự từng bước → trở thành nền móng kỹ thuật cho GPT, BERT và toàn bộ làn sóng LLM sau đó.

2022: ChatGPT



ChatGPT xuất hiện như một trải nghiệm đại chúng

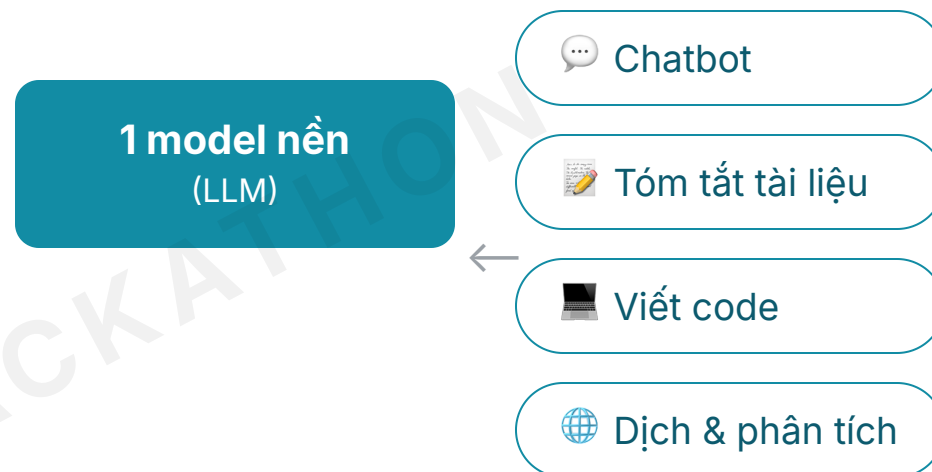
Lần đầu tiên rất đông người dùng phổ thông có thể trực tiếp chạm vào một mô hình ngôn ngữ mạnh, thông qua một giao diện đơn giản đến mức ai cũng hiểu cách dùng

LLM là gì? — một bộ não nền, không phải một chatbot

LLM (Large Language Model) là một mô hình ngôn ngữ rất lớn, thường dựa trên kiến trúc Transformer, được luyện trên hàng nghìn tỷ mảnh chữ để học cách **đoán mảnh chữ tiếp theo trong ngữ cảnh**.

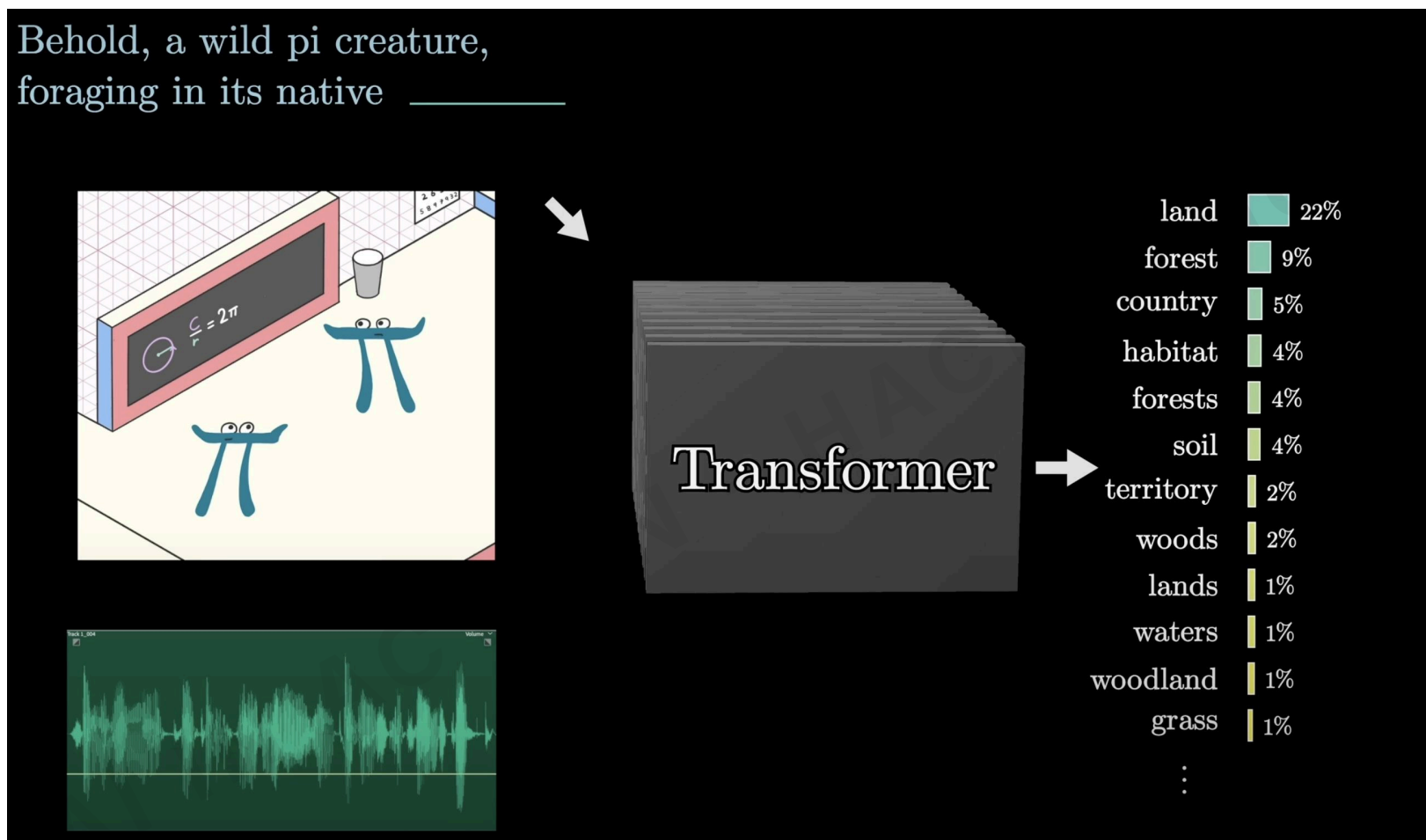
Nhờ được luyện đủ rộng, nó trở thành một **nền chung**: thay vì mỗi việc train một model riêng, cùng một model làm được rất nhiều việc.

Chatbot chỉ là **một dạng sản phẩm** đóng gói quanh bộ não đó — lớp áo bên ngoài.



LLM = bộ não ngôn ngữ dùng chung cho mọi việc — sản phẩm bạn thấy chỉ là lớp áo bên ngoài

Bên trong Transformer: đầu ra luôn là một phân bố xác suất



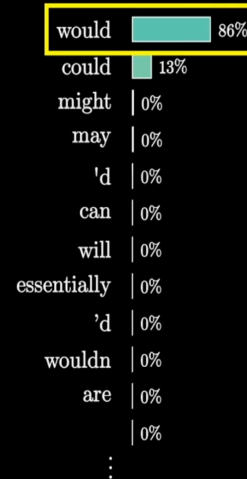
Với mọi ngữ cảnh, model chấm điểm **MỌI** từ trong từ vựng — “land” 22%, “forest” 9%... — rồi chọn theo xác suất đó

Sinh văn bản = đoán → nối vào câu → đoán tiếp

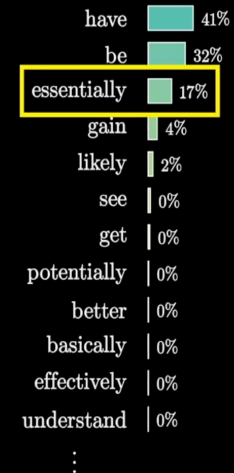
If you could see the underlying probability distributions a large language model uses when generating text, then **you**



If you could see the underlying probability distributions a large language model uses when generating text, then you **would**



If you could see the underlying probability distributions a large language model uses when generating text, then you would **essentially**



Mỗi token mới được nối vào ngữ cảnh, rồi model chạy lại từ đầu — vòng lặp predict → append → rerun

Token: model không đọc "từ", model đọc mảnh chữ

Model không nhìn từ nguyên vẹn. Nó cắt văn bản thành các **mảnh nhỏ gọi là token**: có từ là một mảnh, có từ vỡ ba bốn mảnh, cả dấu câu và khoảng trắng cũng là mảnh.

Ví dụ: *"Hello world"* $\approx$ 2 token, nhưng *"Xin chào"* có thể tới 3–4 token.

Tiếng Việt, code, JSON tốn token hơn tiếng Anh thường — vì dấu thanh, ký tự đặc biệt và cấu trúc bị cắt nhỏ ra.

Độ dài câu XẤP XỈ nhau (9 vs 10 từ) nhưng tokenizer GPT cắt Tiếng Việt vụn hơn hẳn: 1.9 token/từ so với 1.2 token/từ
So sánh KHÔNG cùng số từ tuyệt đối (9 vs 10) — điểm quan trọng là tỉ lệ token/từ, không phải tổng số từ.

Tiếng Anh: "To date, the cleverest thinker of all time was" → 9 từ → 11 token



Tiếng Việt: "Lan bỏ quyển sách vào túi vì nó quá dày" → 10 từ → 19 token



Mình họa bằng tokenizer thật: tiktoken c100k_base (GPT-4). Ô vàng = một "từ" bị cắt thành nhiều token — vd "cleverest" → "clever" + "est"; "Lan" → "L" + "an" (dấu thanh tiếng Việt buộc BPE tách nhỏ hơn vì được huấn luyện chủ yếu trên dữ liệu Anh ngữ).
Nguồn: OpenAI tiktoken (c100k_base), tính trực tiếp trong phiên này.

Mọi thứ model làm đều quy ra token — và mỗi token đều có giá. Nhớ điều này khi sang phần chi phí.

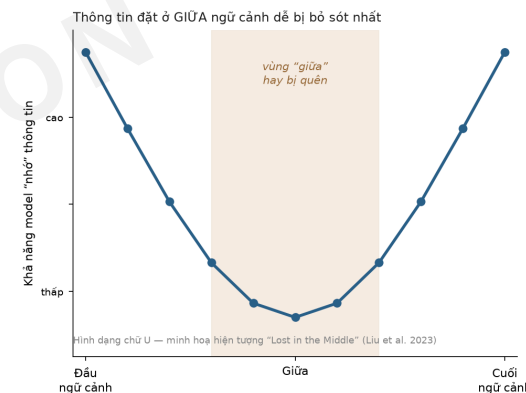
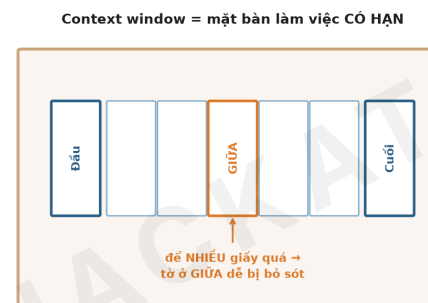
Context: bàn làm việc có hạn của model

Mỗi lần trả lời, model chỉ nhìn được một lượng chữ có hạn — gọi là **context**. Hãy hình dung một **bàn làm việc**: mọi thứ muốn model "thấy" phải bày lên bàn.

Quy đổi: 128K token $\approx$ một cuốn sách 300 trang; 1M token $\approx$ 4–5 cuốn sách trên bàn cùng lúc.

Bàn đầy quá thì đồ ở **giữa bàn dễ bị bỏ sót** — đặt điều quan trọng ở giữa một prompt rất dài, model có thể "quên" mất.

Vì sao "nhỏ" thật nhiều vào prompt không phải lúc nào cũng tốt



Context càng dài càng tốn tiền và càng chậm — bàn rộng không có nghĩa là dùng tốt

Attention: mỗi từ được “nhìn sang” những từ quan trọng khác

Thay vì đọc tuần tự từng chữ, cơ chế **attention** cho phép mỗi token:

- **Chủ động “quay đầu”** nhìn lại các token trước đó trong câu
- **Chấm điểm mức độ liên quan** của từng token đối với nghĩa của mình
- **Khóa nghĩa theo ngữ cảnh** — “nó” là quyển sách hay cái túi, tùy theo nó chú ý vào từ nào

Đây chính là chữ T trong GPT — và là lý do model hiểu ngữ cảnh tốt hơn hẳn các thế hệ trước

Video minh họa: [Attention in transformers, step-by-step - 3Blue1Brown](#)

Hiểu attention để dùng AI hiệu quả: quản context = quản sự chú ý

Attention có hạn và có "điểm mù". Vì vậy, cách bạn bày context quyết định model chú ý vào đâu:

1 Đặt điều quan trọng đầu – cuối

Đầu và cuối prompt được chú ý nhiều nhất; đồ ở giữa dễ bị bỏ sót — yêu cầu quan trọng đừng chôn giữa.

2 Giữ bàn làm việc sạch

Context rác = attention rác. Khi chat dài, tóm tắt lại thay vì kéo theo mọi thứ; khi vibe code, đưa đúng file liên quan, không dán cả repo.

3 Cho tra số thay vì bắt nhớ

Tài liệu dài: lấy đoạn liên quan nhét vào context (RAG) thay vì trông chờ model nhớ hết hoặc nhét cả cuốn.

Agent mạnh không phải vì context khổng lồ — mà vì nó có tools để lấy đúng thứ vào bàn làm việc đúng lúc

Tham số (parameter): những "khớp nối" model học được

Sau khi luyện xong, những gì model "biết" nằm trong các con số cố định bên trong gọi là **tham số** — hãy hình dung như **khớp nối thần kinh**: luyện càng kỹ, các khớp nối càng được siết đúng.

Tham số không phải thứ bạn chỉnh khi dùng model — nó được đóng gói sẵn trong "bộ não" (file weights). Bạn chỉ chỉnh được context và các núm vặn lúc gọi (như temperature).

2020 — GPT-3

175 tỷ

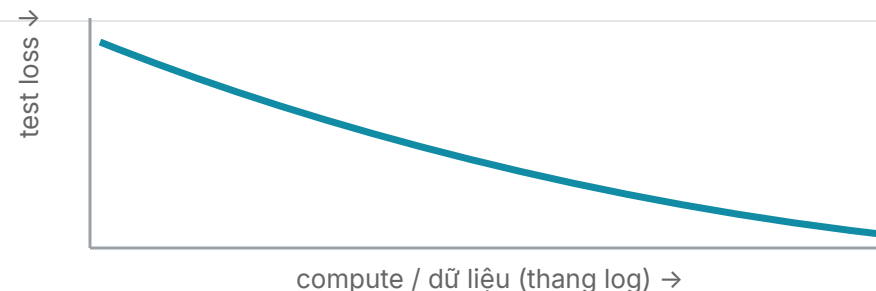
một "bác sĩ đa năng" — mọi token đều đi qua toàn bộ khớp nối (dense)

2026 — Kimi K3

2.800 tỷ

một "bệnh viện đa khoa" — mỗi token chỉ gọi vài chuyên gia (MoE)

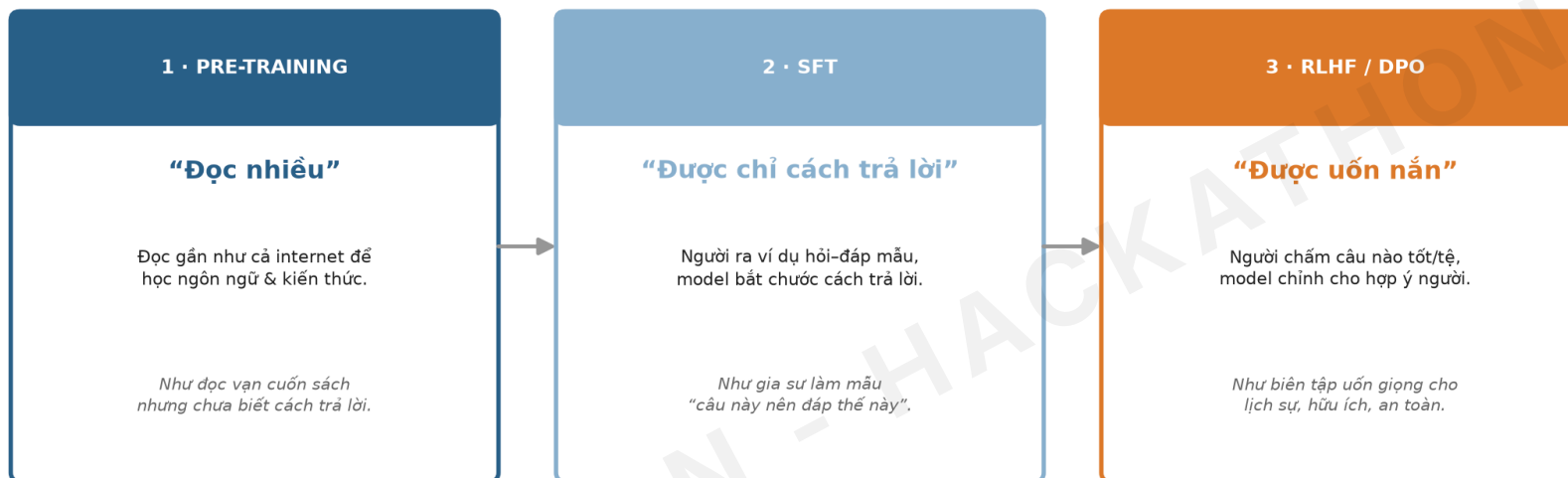
Nhiều tham số ≠ tốn hơn tuyến tính — nhờ MoE, bệnh viện lớn gấp 16 lần mà chi phí mỗi ca khám gần như không đổi



Luật chơi 2020–2024: cứ thêm compute + dữ liệu là model khôn lên
một cách dự đoán được (scaling law, Kaplan et al. 2020)

LLM được tạo ra như thế nào? — đọc nhiều, được chỉ, được uốn nắn, luyện đề

Một LLM được “nuôi lớn” qua 3 bước — mỗi bước một việc khác nhau

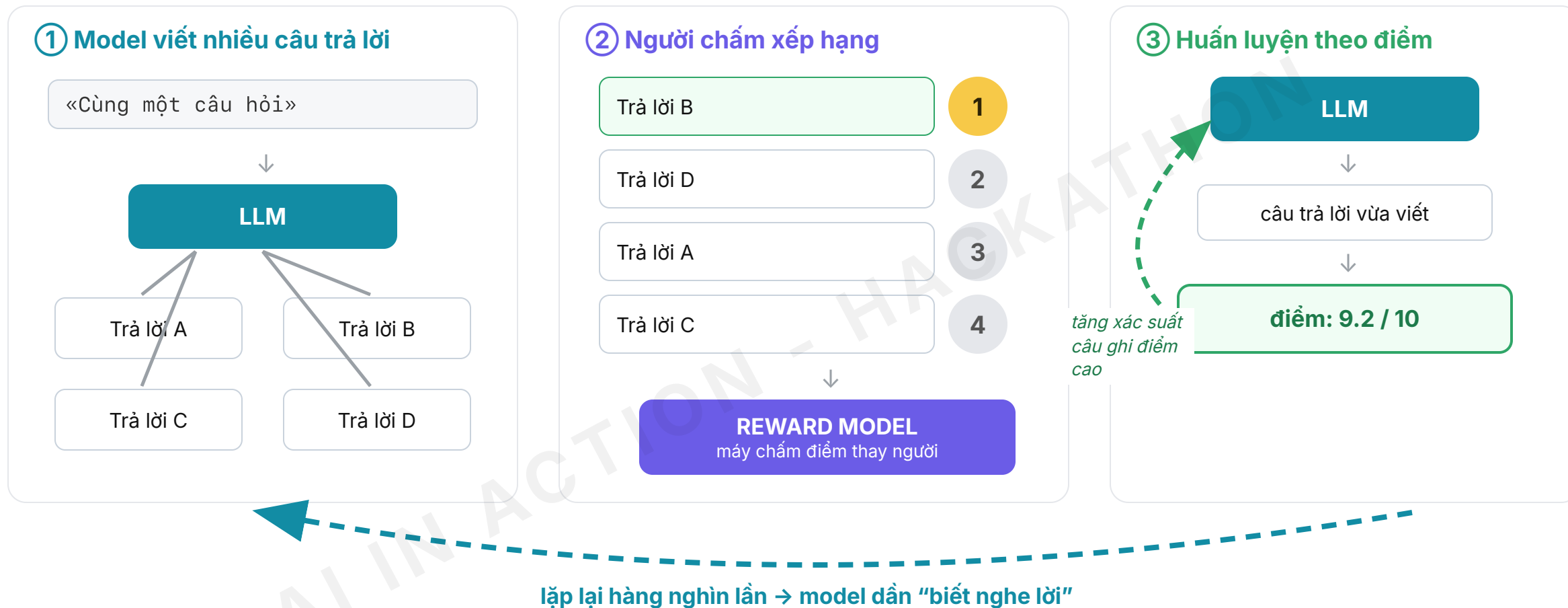


Ẩn dụ “học giả trong bong bóng”: đọc rất nhiều (bước 1) nhưng phải được chỉ (2) và uốn nắn (3) mới biết đối đáp cho ra người.

① **Pre-training** — “**đọc cả thư viện**”: học tiếng nói và kiến thức từ hàng nghìn tỷ token. ② **SFT** — “**được chỉ cách trả lời**”: học theo ví dụ mẫu để ra đáng trợ lý. ③ **RLHF/DPO** — “**được uốn nắn**”: học theo phản hồi con người, an toàn và dễ chịu hơn. ④ **Luyện suy luận** — “**giải đề tự chấm**” (từ ~2025): luyện toán/code có đáp án kiểm chứng được → model biết làm nháp trước khi trả lời.

Đọc vạn cuốn sách chưa chắc biết trả lời phỏng vấn — đó là lý do cần bước ②, ③, ④

RLHF: ba bước uốn cổ máy đoán token thành trợ lý biết nghe lời



Cỗ máy đoán token + điểm xếp hạng của con người → trợ lý helpful · harmless · honest

Giới hạn bẩm sinh: học giả trong bong bóng

Bong bóng thời gian

Model bị "đóng băng" tại ngày ngừng đọc. Chuyện sau đó nó không biết — trừ khi bạn cung cấp thêm (knowledge cutoff).

Nói chắc như đúng rồi

Model tối ưu cho câu **nghe hợp lý**, không phải tra sự thật — nên có thể tự tin mà sai (hallucination).

Bản làm việc có hạn

Context có trần; quá dài vừa tốn tiền vừa dễ bỏ sót thông tin ở giữa.

"Why does it work? We don't know — a lot here are intuitions, not theorems or truths." — Łukasz Kaiser, đồng tác giả
"Attention Is All You Need" (OpenAI)

*Đây không phải lỗi tạm thời — đó là **bản chất của cỗ máy đoán token**. Vì vậy ta cần prompt tốt, context sạch, tra số (RAG), tools, và luôn kiểm chứng.*

Vì sao model vẫn sai: nó rất giỏi học vẹt đường tắt

1 Phân loại spam

Model thực chất đã học:

"đếm số hyperlink trong email"

Email sạch nhưng nhiều link → vẫn bị gán spam

2 Câu chủ quan vs khách quan

Model thực chất đã học:

"có phải câu trích từ film review không"

Ăn gian bằng nguồn gốc câu, không phải nội dung câu

3 Suy luận ngôn ngữ (MNLI)

Model thực chất đã học:

"câu có động từ phủ định"

Đổi cấu trúc dữ liệu test là điểm tụt ngay

Ba "đường tắt" (spurious cues) trên do **chính LLM tự động phát hiện** và mô tả bằng ngôn ngữ tự nhiên — trên quy mô 675 bài toán thật của benchmark OpenD5.

Benchmark cao ≠ model hiểu đúng thứ bạn tưởng — luôn test trên dữ liệu của chính mình

Chain-of-Thought: chỉ thêm "giấy nháp", từ sai thành đúng

Bài toán: "Có 5 quả bóng tennis. Mua thêm 2 hộp, mỗi hộp 3 quả. Hỏi tổng cộng có bao nhiêu quả?"

Không có nháp — trả lời ngay

Model đọc câu hỏi → bật ra đáp án ngay:

"Đáp án là 27 quả."

× SAI

Có giấy nháp — "hãy nghĩ từng bước"

"Bắt đầu có 5 quả.

Mỗi hộp 3 quả × 2 hộp = 6 quả.

$5 + 6 = 11$.

Đáp án là 11 quả."

✓ ĐÚNG

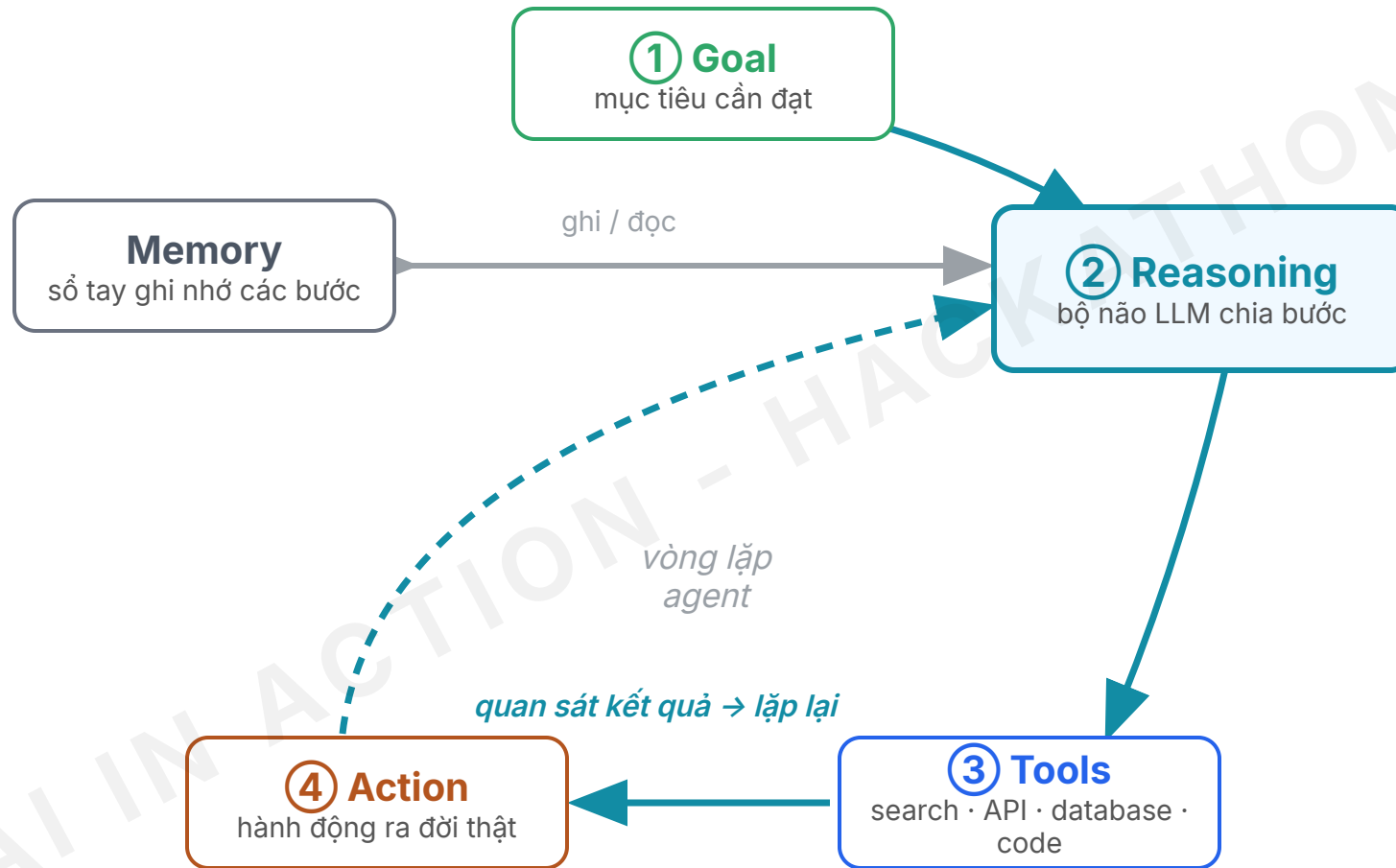
Cùng một model, cùng một câu hỏi — cho nó được viết nháp từng bước, bản chất suy luận lộ ra

Từ LLM đến agent: bốn mức độ — mỗi bậc thêm một năng lực



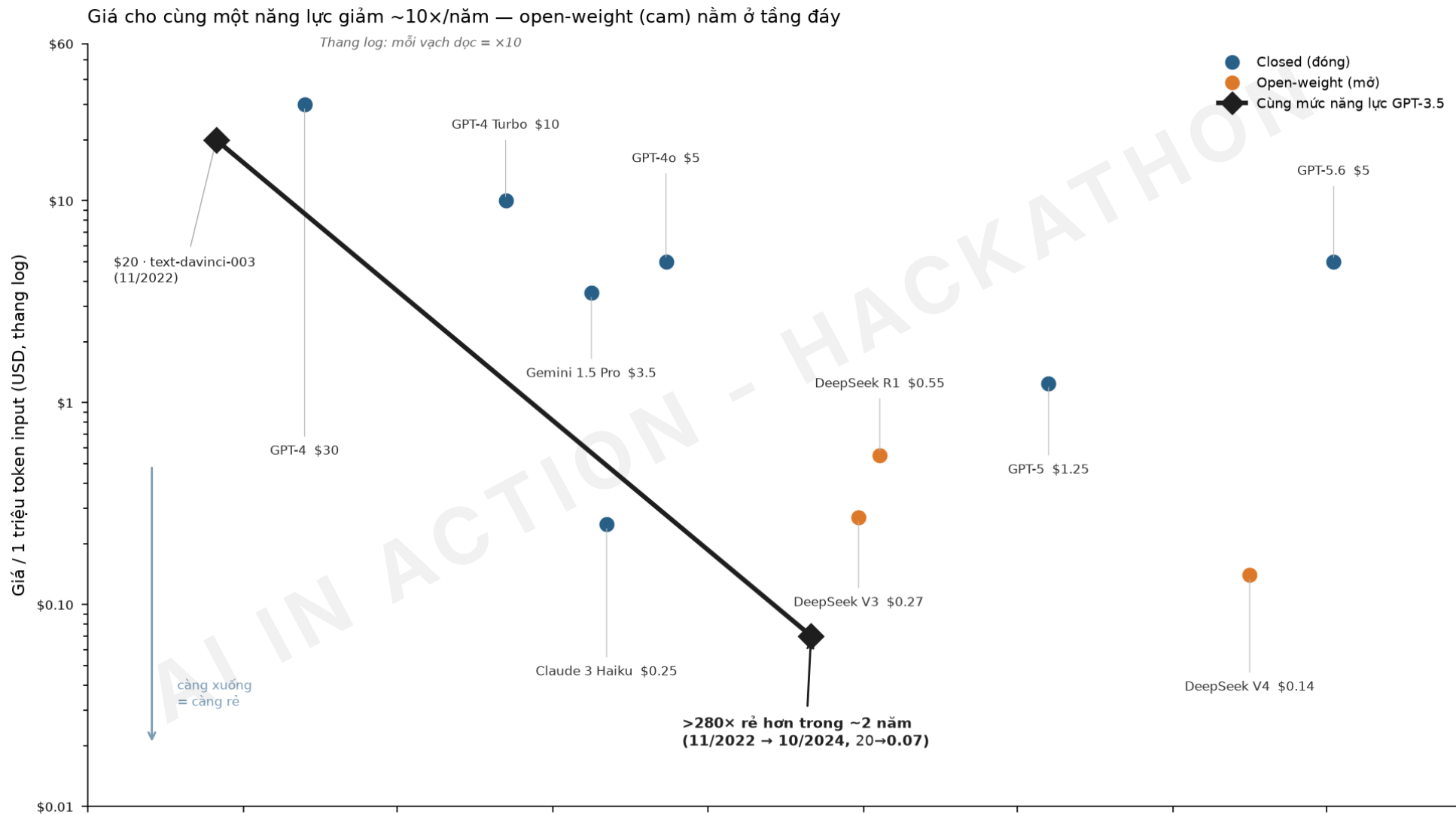
Agent không phải “một loại model khác” — đó là LLM được đặt vào vòng làm việc có mục tiêu và hành động

Giải phẫu một agent: 5 bộ phận là một vòng lặp



Agent = Goal + Reasoning + Tools + Memory + Action — chạy thành vòng lặp cho tới khi xong việc

Cùng một mức năng lực, giá rơi khoảng 10 lần mỗi năm



Việc năm ngoái phải dùng model đắt nhất — năm nay model rẻ đã làm được

Tổng hợp từ bảng giá các nhà cung cấp, 2023–2026.

Chọn model theo TẦNG, không chọn theo tên

VIỆC CỦA BẠN

Việc đơn giản, khối lượng lớn

phân loại · trích xuất · tóm tắt ngắn

Việc hàng ngày

viết · code · phân tích công việc · automation

Việc khó nhất

suy luận nhiều bước · code phức tạp · tài liệu dài · độ tin cậy cao

Việc cần kiểm soát

dữ liệu nhạy cảm · chi phí ở quy mô lớn

TẦNG MODEL

TẦNG 1 — FRONTIER ĐÓNG

Fable 5 · GPT-5.6 Sol · Opus 4.8

đắt nhất — chỉ trả cho việc thật sự khó

★ MẶC ĐỊNH THỬ TẦNG NÀY TRƯỚC

TẦNG 2 — RẺ MÀ MẠNH

Sonnet 4.6 · Terra · Gemini 3.1 Pro · Kimi K3 · Haiku · Flash

giải quyết đa số việc hàng ngày

TẦNG 3 — SELF-HOST / SIÊU RẺ

Kimi K3 open-weight · DeepSeek · Qwen

khi cần kiểm soát dữ liệu hoặc chi phí quy mô lớn

Hai lỗi đối xứng:

- × việc đơn giản mà gọi frontier → phí tiền
- × việc khó mà cố dùng rẻ → kết quả tệ

Bắt đầu từ model đủ tốt và đủ rẻ — chỉ nâng tầng khi kết quả thực sự chẵn use case

Token có giá: vé vào rẻ, vé ra đắt gấp 3–5 lần

VÉ VÀO — INPUT

×1

chữ BẠN gửi đi:
prompt · system instruction ·
context · lịch sử chat

rẻ — model chỉ cần đọc

VÉ RA — OUTPUT

×3–5

chữ MODEL viết ra — nó phải
tự sinh từng mảnh một, vừa
chậm vừa tốn

đắt — model phải “vất óc”

HÓA ĐƠN — 1 LẦN GỌI API

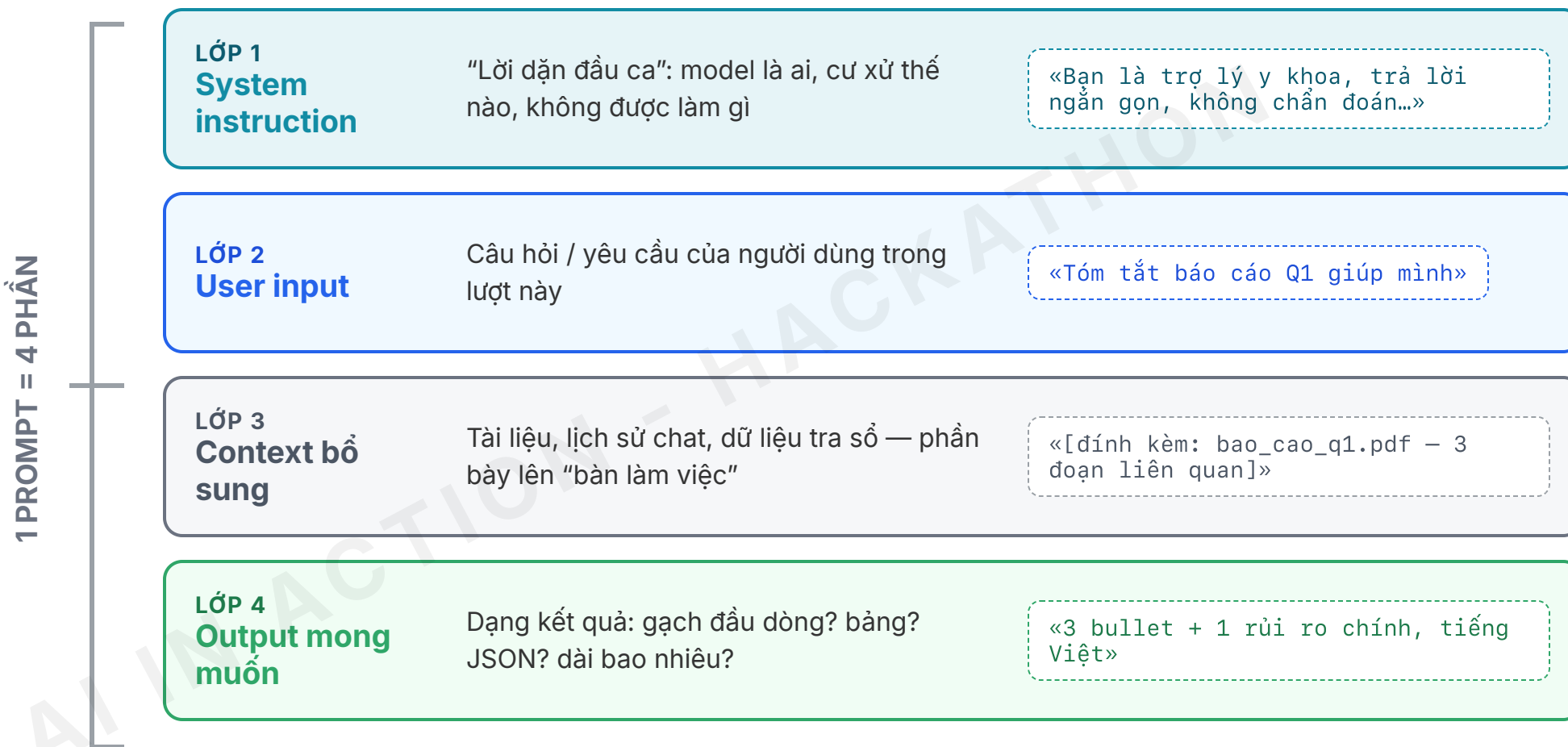
input	1.150 tok × \$3 / 1M	\$0.00345
output	200 tok × \$15 / 1M	\$0.00300
TỔNG		≈ \$0.0065

số liệu ví dụ — giá thật tùy model & nhà cung cấp

Đọc mục **usage** trong mỗi response — đó là hóa đơn chi tiết giúp bạn kiểm soát chi phí từ ngày đầu.

Input tokens + Output tokens = Chi phí mỗi lần gọi — kiểm soát output là nôm vắn lớn nhất

Giải phẫu một prompt: bốn lớp xếp chồng



Viết rõ cả 4 lớp = đã làm tốt một nửa "prompt engineering" — phần còn lại là các ngày sau

Hai núm vặn chọn từ: temperature & top_p

temperature — “núm vặn độ liều”

Cùng một câu: “Một tách ___” — bảng xác suất đổi theo T

T = 0



luôn chọn từ **chắc nhất**
→ ổn định, lặp lại, hợp
code & phân tích

T = 1



cân bằng tự nhiên —
vẫn ưu tiên từ hợp lý

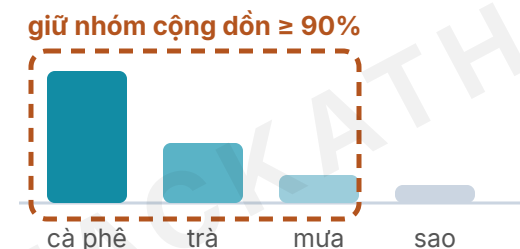
T = 2



phân bố **phẳng ra** →
đa dạng, “phiêu”, dễ
lạc đề

top_p — “chỉ xem top đầu bảng” (p = 0.9)

① Bảng xác suất gốc



② Bảng mới



“sao” (đuôi dài xác suất thấp) **bị loại khỏi lựa chọn** — model chỉ còn chọn trong nhóm đáng tin. Thường chỉ vận **một trong hai**: temperature *hoặc* top_p.

Lưu ý quan trọng: hai núm này không làm model thông minh hơn — chỉ đổi **cách chọn từ**, không thêm tri thức.

Mặc định an toàn: temperature = 0 cho việc cần ổn định — chỉ tăng khi thật sự cần đa dạng